

1. INFIX TO POSTFIX CONVERSION PSEUDOCODE

BEGIN

CREATE empty stack S

CREATE empty output list POSTFIX

WHILE there are tokens in the infix expression DO

READ next token T

IF T is an operand THEN

APPEND T to POSTFIX

ELSE IF T is '(' THEN

PUSH T onto stack S

ELSE IF T is ')' THEN

WHILE top of S is not '(' DO

POP from S and APPEND to POSTFIX

END WHILE

POP '(' from S // discard it

ELSE IF T is an operator THEN

WHILE S is not empty AND

top of S is an operator AND

(precedence (top of S) > precedence(T) OR

(precedence (top of S) = precedence(T)

AND T is left associative)) DO

POP from S and APPEND to POSTFIX

END WHILE

PUSH T onto S

END IF

END WHILE

// After all tokens processed

WHILE S is not empty DO

POP from S and APPEND to POSTFIX

END WHILE

OUTPUT POSTFIX expression

END

2. INFIX TO PREFIX CONVERSION WORKFLOW

BEGIN

REVERSE the infix expression

SWAP '(' with ')' and ')' with '(' in the reversed expression

CREATE empty stack S

CREATE empty output list PREFIX_TEMP

WHILE there are tokens in the modified expression **DO**

READ next token T

IF T is an operand **THEN**

APPEND T to PREFIX_TEMP

ELSE IF T is '(' **THEN**

PUSH T onto stack S

ELSE IF T is ')' **THEN**

WHILE top of S is not '(' **DO**

POP from S and **APPEND** to PREFIX_TEMP

END WHILE

POP '(' from S // discard it

ELSE IF T is an operator **THEN**

WHILE S is not empty **AND**

 top of S is an operator **AND**

OR

AND T is left associative)) **DO**

POP from S and **APPEND** to PREFIX_TEMP

END WHILE

PUSH T onto S

END IF

END WHILE

// After all tokens processed

WHILE S is not empty **DO**

POP from S and **APPEND** to PREFIX_TEMP

END WHILE

REVERSE PREFIX_TEMP to get PREFIX

OUTPUT PREFIX expression

END